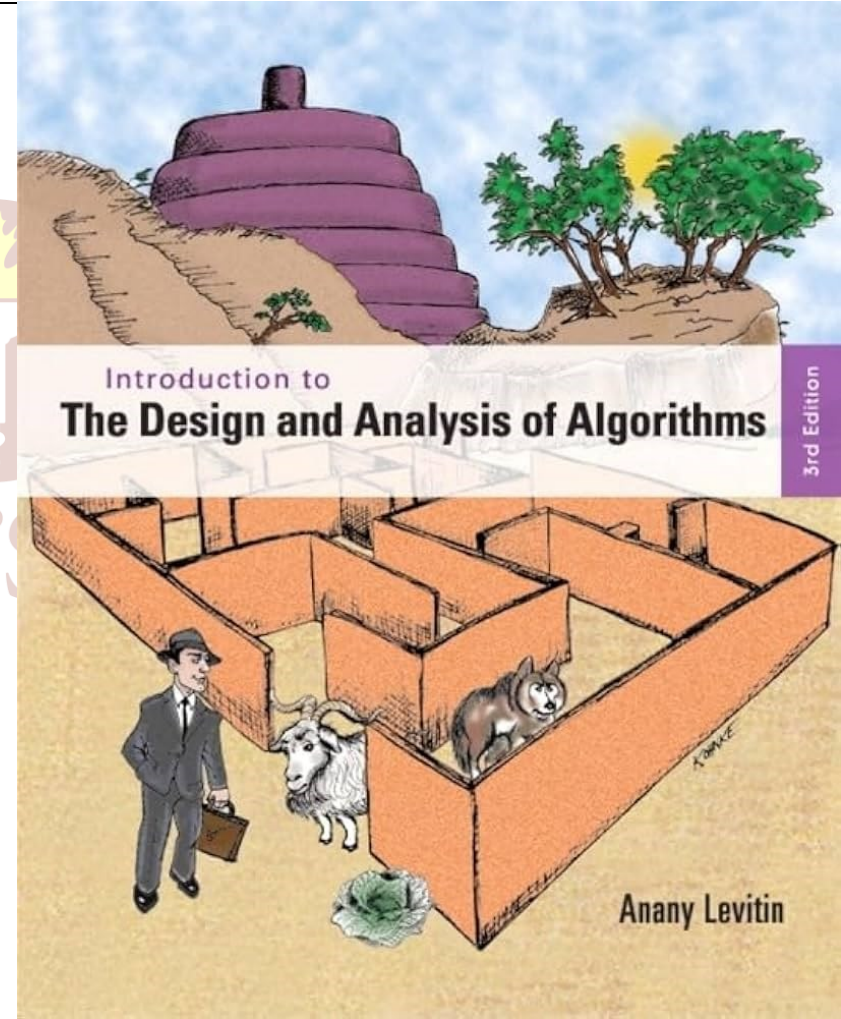


CS 07340 DESIGN AND ANALYSIS OF ALGORITHMS

WEEK 4 OVERVIEW

- Decrease-and-Conquer
 - Insertion Sort
 - Binary Search (in sorted array)
 - Binary Search Tree (BST)



DECREASE-AND-CONQUER

Decrease-and-conquer is related to the ***divide-and-conquer*** we will look at next, but they are different in complexity.

Decrease-and-conquer involves reducing the problem into **one** smaller subproblem, while divide-and-conquer reduces the problem into **several** similar subproblems. The main idea will be to

1. **Decrease** / reduce the problem instance to a smaller instance of the same problem.
2. **Conquer** the problem by solving the smaller.
3. **Extend** the solution of the smaller instance back to the original problem.

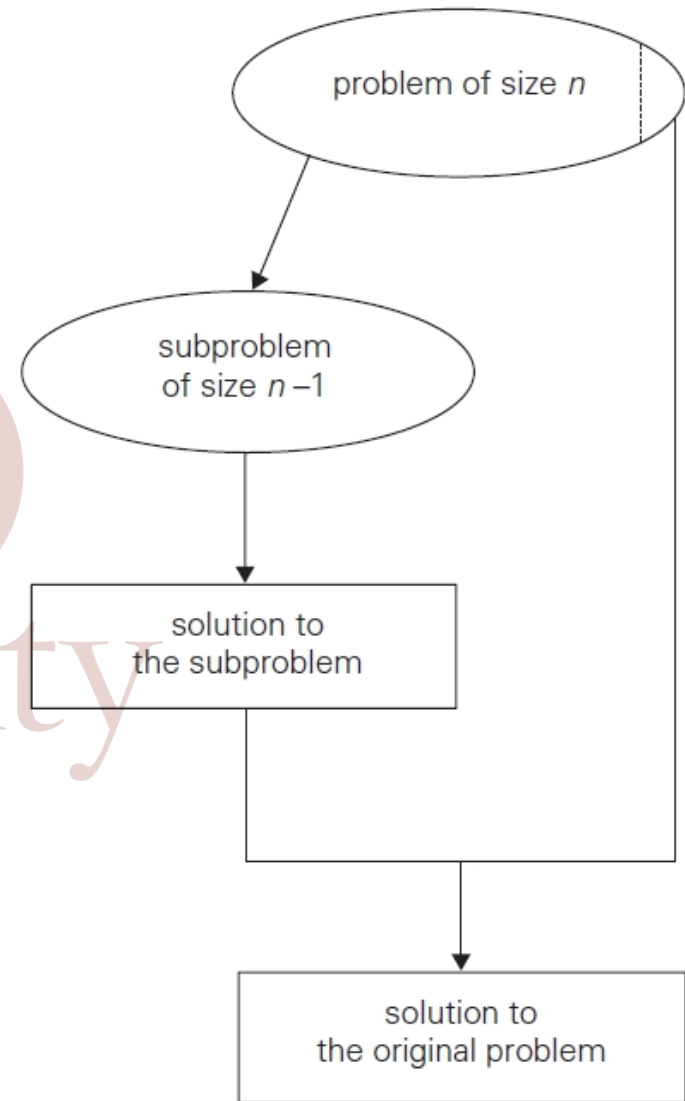
There are three major variations of decrease-and-conquer:

- decrease by a constant
- decrease by a constant factor
- variable size decrease



DECREASE-BY-A-CONSTANT

We know some examples of ***decrease-by-a-constant*** by the name of tail-recursion problems: exponentiation in a loop, such as $a^n = a^{n-1} \cdot a$, or calculating $n!$ as $(n-1)! \cdot n$.



INSERTION SORT

Insertion Sort is a [simple sorting algorithm](#) that is based on a recursive idea: Sorting an array with n elements when the leading array with $n - 1$ elements is already sorted. We only need to find the correct place for the new element. This can be easily implemented non-recursively in a double-loop. In fact, [Jon Bentley](#) has a [three-line implementation](#) (see 4.1 Exercise 10 in our book):

```
for i = [1, n)
  for (j = i; j > 0 && x[j-1] > x[j]; j--)
    swap(j-1, j)
```

Hence, we start with an array of size 1, increasing the size by 1 each iteration, and find the correct place for the new element.

ALGORITHM *InsertionSort*($A[0..n - 1]$)

//Sorts a given array by insertion sort

//Input: An array $A[0..n - 1]$ of n orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

for $i \leftarrow 1$ **to** $n - 1$ **do**

$v \leftarrow A[i]$

$j \leftarrow i - 1$

while $j \geq 0$ **and** $A[j] > v$ **do**

$A[j + 1] \leftarrow A[j]$

$j \leftarrow j - 1$

$A[j + 1] \leftarrow v$



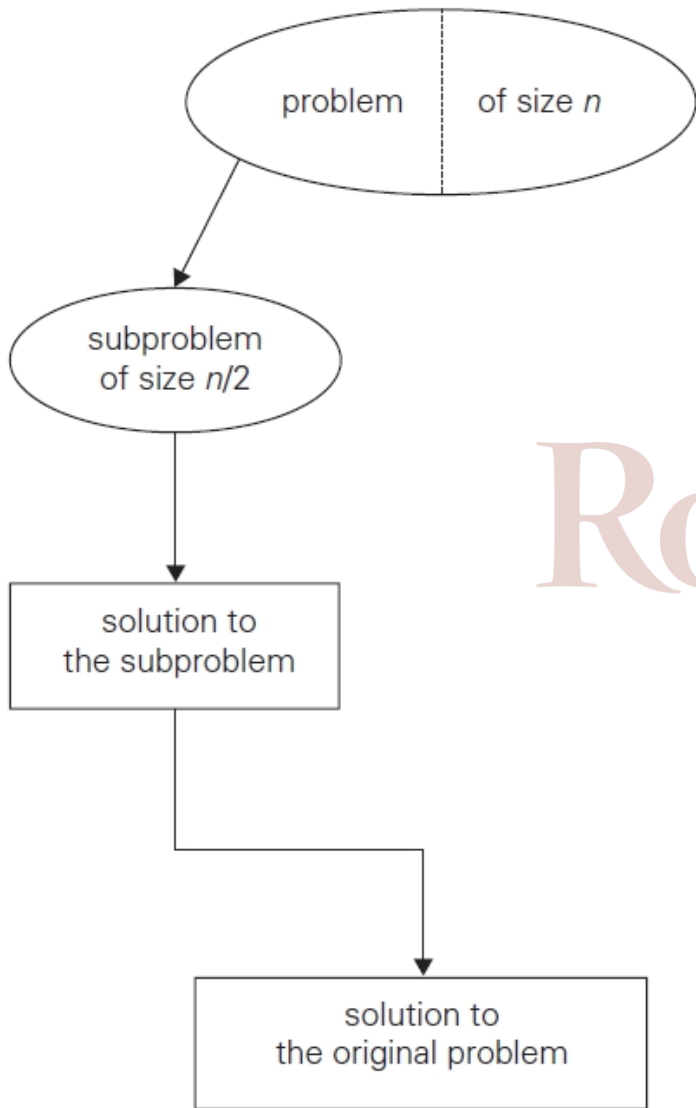
Example: Sort the array {89, 45, 68, 90, 29, 34, 17} by hand using *Insertion Sort*.

Programming: Create a sample implementation for *Insertion Sort* in Java.

Insertion Sort is $\Theta(n^2)$ in worst case (and $\Theta(n)$ in best case). It is an in-place sorting algorithm.

Example: Write pseudo code for a factorial function as a tail end recursion.





DECREASE-BY-A-FACTOR

If we compute a^n recursively by

$$a^n = \begin{cases} (a^{n/2})^2 & \text{if } n \text{ is even and positive} \\ (a^{(n-1)/2})^2 \cdot a & \text{if } n \text{ is odd} \\ 1 & \text{if } n = 0 \end{cases}$$

Then we should expect the algorithm to be $\Theta(\log(n))$.

BINARY SEARCH

Binary Search is closely related to the idea of (balanced) binary search trees. It is an application of decrease-by-a-factor. [Binary search](#) compares the target key value to the middle element (which has the **median value**) of the array. Equality comparisons can be implemented as a [three-way-comparison](#) (Java [comparator](#)):

- is equal
- is less than
- is larger than

If the target value is equal to the mean, we found our target. If it is less, it must be in the left half of the array, otherwise in the right half. We can repeat this process until we find our target.

The [Bisection Method](#) from Calculus uses the same idea.

ALGORITHM *BinarySearch*($A[0..n - 1]$, K)

//Implements nonrecursive binary search

//Input: An array $A[0..n - 1]$ sorted in ascending order and

// a search key K

//Output: An index of the array's element that is equal to K

// or -1 if there is no such element

$l \leftarrow 0$; $r \leftarrow n - 1$

while $l \leq r$ **do**

$m \leftarrow \lfloor (l + r)/2 \rfloor$

if $K = A[m]$ **return** m

else if $K < A[m]$ $r \leftarrow m - 1$

else $l \leftarrow m + 1$

return -1

Example: Use Binary Search to find key 70 in the ordered array

3	14	27	31	39	42	55	70	74	81	85	93	98
---	----	----	----	----	----	----	----	----	----	----	----	----

Programming: Implement a Binary Search algorithm in Java.

Binary Search has a runtime of $O(\log(n))$. Note that for Binary Search to work, the array had to be sorted. The best sorting algorithms for an array are $\Theta(n \cdot \log(n))$, so some work had to be done first for the search to be efficient.

VARIABLE-SIZE-DECREASE

$$\text{gcd}(m, n) = \text{gcd}(n, m \bmod n)$$

decreases arguments, but neither in a constant nor percentage style.

An example of variable-size-decrease is search in a generic *Binary Search Tree*. The problem of finding a key is reduced to a smaller problem each time we make a comparison, since the left or right subtree that we need to traverse **will be smaller**. We just don't know by how much.

In contrast, search in a ***balanced BST*** is an example of ***divide-and-conquer***. In a weight-balanced BST, left and right subtrees have about the same number of elements.

QUIZ TOPICS

- Decrease-and-conquer algorithm examples
 - Tail end recursion
 - Binary Search
- Runtime analysis of decrease-and-conquer algorithms

